Modificación del DOM: Poder y Peligro

Buenas prácticas, atajos tóxicos y prevención de XSS en JavaScript

NotebookLM

El paradigma de los nodos: Construcción estructural

Paso 1:

```
document.createElement('p')
```

→ `<p>`

Paso 2:

```
document.createTextNode('Hola')
```

→ Hola

Paso 3:

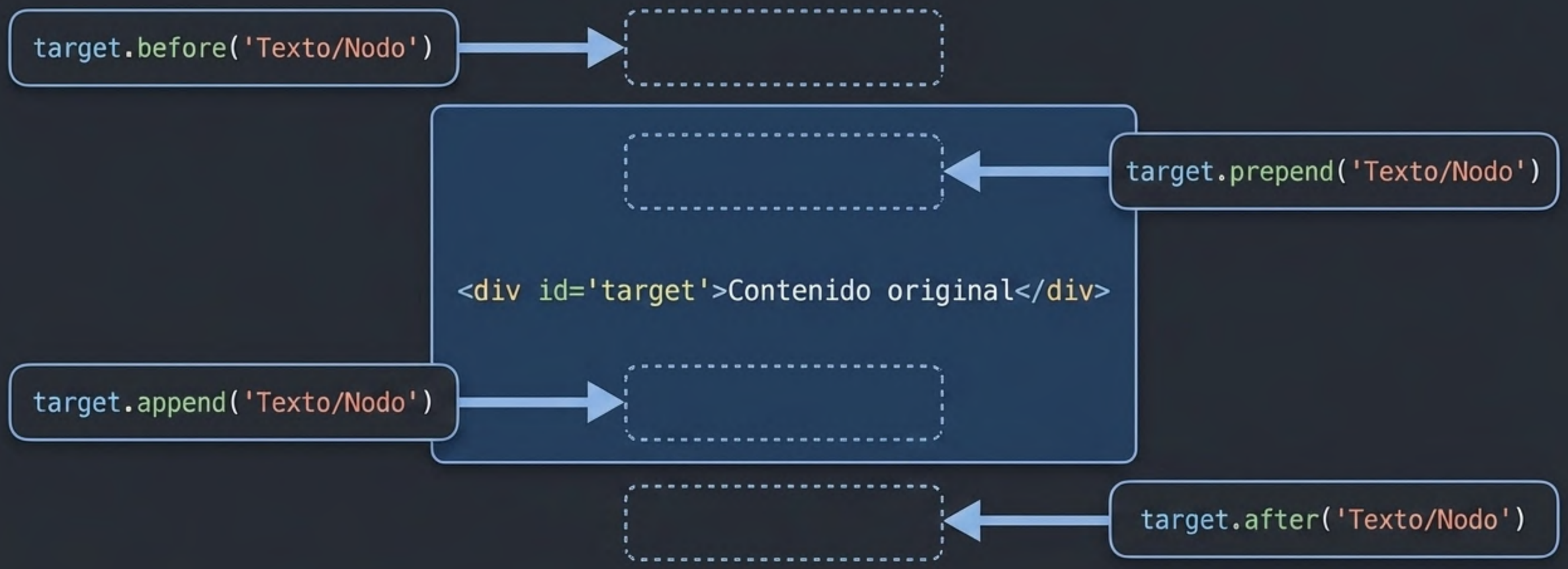
```
elemento.appendChild(nodo)
```

→ Hola

El navegador construye el árbol DOM paso a paso. El contenido se procesa estrictamente como datos inertes, garantizando la seguridad estructural.

NotebookLM

Topología moderna de inserción



Los métodos modernos permiten insertar múltiples nodos o cadenas de texto simultáneamente.
Las cadenas se insertan de forma segura, sin riesgo de ejecución de código.

NotebookLM

La tentación del atajo directo

El camino largo

```
const div =
  document.createElement('div');
div.className = 'alert';
div.textContent = '¡Hola!';
elemento.append(div);
```

El atajo

```
elemento.innerHTML =
"<div class='alert'>¡Hola!</div>";
```

Los desarrolladores prefieren escribir HTML como texto. El navegador analiza la cadena y crea los nodos automáticamente. Es rápido, elegante... y altamente peligroso si se usan datos no confiables.

NotebookLM

La anatomía de una vulnerabilidad crítica

Entrada del Usuario

Un atacante teclea código malicioso en un formulario o manipula la URL.

La Mala Práctica

```
elemento.innerHTML
= userInput;
```

Motor del Navegador

El navegador recibe la cadena y la ejecuta. No distingue entre el marcado intencional y el ataque inyectado.

ADVERTENCIA: Pasar entrada de usuario directamente a métodos de análisis HTML (como innerHTML o insertAdjacentHTML) es la causa principal de inyecciones XSS en el frontend.

NotebookLM

Cross-Site Scripting (XSS) en acción

```
<img src='x' onerror='alert("Hackeado")'>>
```

Enlace roto intencional. Fuerza un error de carga inmediato en la imagen.

Vector de ataque. Evento del DOM que ejecuta JavaScript automáticamente sin interacción.

Payload malicioso. Se ejecuta en el contexto de la aplicación, permitiendo robo de datos.

No se requiere una etiqueta `<script>`. Los vectores basados en eventos ejecutan código de manera instantánea durante el proceso de renderizado del HTML.

NotebookLM

El peligro del doble contexto (JS + HTML)

El Escenario El desarrollador inserta datos escapados en un atributo de evento.

```
<button onclick='var data ="[ENTRADA_USUARIO]"'>Click</button>
```

El Ataque Entrada inyectada: `"; alert("XSS");//"`

```
<button onclick='var data =""; alert("XSS");//"'>Click</button>
```

Codificar entidades HTML no es suficiente dentro de contextos de script. La comilla inyectada cierra la variable prematuramente, el **punto y coma** inicia un nuevo comando ejecutable, y las barras comentan el resto del código inofensivo.

NotebookLM

La regla de oro para cadenas: `textContent`

```
<script>alert('XSS')</script>
```

Entrada No Segura

```
elemento.textContent
 = userInput;
```

Procesamiento Seguro

```
<script>alert('XSS')</script>
```

Salida Literal

El navegador trata la entrada estrictamente como una cadena literal plana. Los caracteres '<' y '>' se escapan automáticamente en el motor de renderizado, despojando al texto de cualquier significado estructural o ejecutable.

NotebookLM

Ejecución vs. Texto Plano

Entrada del usuario: "Hacker"

La Mala Práctica

```
elem.innerHTML = userInput;
```

Hacker

El navegador procesó y ejecutó la etiqueta HTML.

El Estándar Seguro

```
elem.textContent = userInput;
```

Hacker

El navegador neutralizó la etiqueta, mostrándola como texto inerte.

NotebookLM

Inserción de HTML dinámico de forma segura

Entrada de usuario (Texto Enriquecido)

```
<b>Texto seguro</b> <img src=x onerror=alert("XSS")>
<script>malicious()</script>
```

Procesamiento de Sanitización

```
const htmlSeguro = DOMPurify.sanitize(userInput);
```

Salida Segura

```
<b>Texto seguro</b>
```

```
elemento.innerHTML = htmlSeguro;
```

Si el uso de innerHTML es indispensable (ej. editores de texto), NUNCA confíes en los datos recibidos. recibidos. Procesa siempre el código a través de una librería de sanitización dedicada.

NotebookLM

APIs heredadas y obsoletas

```
document.write('<h1>Hola</h1>');
```

Pertenece a la era anterior a los estándares modernos del DOM. No debe usarse en aplicaciones actuales.

Si se invoca de forma asíncrona después de que la página ha cargado, borra por completo el documento existente.

Bloquea la canalización de renderizado del navegador y representa un riesgo de seguridad de arquitectura masivo.

NotebookLM

Matriz de decisión de manipulación del DOM

Método	Convierte a HTML	Riesgo XSS	Veredicto
append() / createElement()	No	Bajo	Seguro pero verboso
textContent	No	Nulo	El escudo predeterminado
innerHTML / insertAdjacentHTML()	Sí	Crítico	Solo con sanitización
document.write()	Sí	Crítico	Evitar siempre

NotebookLM

Manifiesto de seguridad del DOM

[OK]	Respeta la arquitectura:	Prefiere la construcción de nodos y la topología moderna sobre la inserción inestable de cadenas de texto.
[OK]	Usa el escudo predeterminado:	Utiliza 'textContent' estrictamente para inyectar, actualizar y mostrar cualquier texto plano en la interfaz.
[OK]	Política de cero confianza:	Nunca pases datos de usuario crudos a métodos de evaluación sintáctica o renderizado directo.
[OK]	Sanitización obligatoria:	Si el procesamiento de HTML dinámico es absolutamente vital, fitra siempre la entrada a través de un sanitizador dedicado y robusto.

NotebookLM